

Blatt 6

Kryptologie

Luis Staudt

ECDSA-signierte Kassenbons (TSE-Signatur)

Aufgabenstellung

Die Technische Sicherheitseinrichtung (TSE) signiert jeden Kassenvorgang mit ECDSA. Die signaturrelevanten Daten stehen im QR-Code des Bons. Ohne App soll die Gültigkeit der angegebenen TSE-Signatur *von Hand* nachgerechnet werden:

- a) Inhalt des QR-Codes verifizieren (Felder zuordnen, r , s und Public Key D dekodieren).
- b) Zugehörige LOG-MESSAGE gemäß BSI TR-03151 aufstellen und Hashwert berechnen.
- c) Passende elliptische Kurve E ermitteln, indem geprüft wird, ob $D \in E$ liegt.
- d) Gültigkeit des Unterschriftspaares (r, s) verifizieren.
- e) Verifikation eines eigenen (hier selbst erzeugten) Bons.

Verwendete Werkzeuge

- **Java** mit den JDK-Klassen `MessageDigest` (SHA-256/SHA-384), `Base64` und `java.time.Instant` für Dekodierung, Aufbau der Log-Message sowie Hash- und Zeitstempelberechnung.
- Die elliptische Kurven-Arithmetik (Punktaddition, skalare Multiplikation, Punkt-auf-Kurve-Test) übernimmt die erprobte Bibliothek **Bouncy Castle** (`org.bouncycastle.math.ec`); die benannten Kurvenparameter (`brainpoolP384r1`, `secp384r1`) stammen aus `ECNamedCurveTable`.
- Programm: `src/net/luis/task6/TseSignatureVerification.java`.

a) Inhalt des QR-Codes

Der QR-Code ist eine durch ; getrennte Liste fester Felder. Die Zuordnung zur BSI-Spezifikation lautet:

Feld (BSI)	Wert im QR-Code
version	V0
clientID (Kasse)	P915560150
processType	Kassenbeleg-V1
processData	Beleg^0.00_1.65_0.00_0.00_0.00^1.65:Bar
transactionNumber	124226
signatureCounter	387826
Start-Zeit	2022-10-31T06:29:44.000Z
Log-/End-Zeit	2022-10-31T06:29:58.000Z
signatureAlgorithm	ecdsa-plain-SHA384
logTimeFormat	unixTime
Signatur (r, s)	Base64 (96 Byte)
Public Key D	Base64 (97 Byte)

Tabelle 1: Zuordnung der QR-Code-Felder.

Signatur. Die Base64-Signatur dekodiert zu genau 96 Byte. Bei *ecdsa-plain* ist dies die Konkatination der beiden 48-Byte-Werte $r \parallel s$ ($2 \cdot 384$ Bit):

```

1 r = 6FA9BBAF63C0944AE393D737D1CE1B90F8ABA6A372DB486CE352EB26
2   ECA876EA1D63BDCF0750FA26F411AD1359FA3E0F
3 s = 34C182A251737943CED3B256CD091742A3BDC43F429427799D77589F
4   F051673D6306F76321E32370E2F2EFD2B4B0806

```

Code Listing 1: Dekodiertes Signaturpaar (r, s) (hex)

Public Key. Die Base64-Daten dekodieren zu 97 Byte. Das erste Byte ist $0x04$ (unkomprimierter Punkt), gefolgt von $D_x \parallel D_y$ zu je 48 Byte:

```

1 Dx = 18991AC866345CD711FD074AF239ED2869899E9CDA6DCD3CA555A5B8
2   931DC36F80AD2012042009FDCF30CBD9073EF6E4
3 Dy = 071DC32724E40A4FDD88E6A22181306A3331210B1E76A8AAFBE7AC13
4   4B2C9B4BE9F7898DB7E530678C853F353785A1B

```

Code Listing 2: Dekodierter Public Key $D = (D_x, D_y)$ (hex)

Plausibilitätsprobe. Die Seriennummer der TSE ist der SHA-256-Hash des Public Key (inkl. Tag $0x04$). Berechnet ergibt sich

```

1 serialNumber = SHA256(04 || Dx || Dy)
2               = 2DE151380B91213FBADFAF18EF856DB499E608283
3               = 984F96297213F450BA552B7

```

Dieser Wert stimmt exakt mit der auf dem Bon aufgedruckten TSE-Seriennr. 2DE151380B91213FBADFAF18EF856DB499E608283... überein. Damit ist der QR-Inhalt konsistent.

b) Log-Message und Hashwert

Die LOG-MESSAGE M wird gemäß BSI TR-03151 als ASN.1/DER im Tag-Length-Value-Format aus den QR-Feldern zusammengesetzt. Verwendete Felder in Reihenfolge:

Feld	Tag	Wert (kodiert)
version	02	02 (=2)
certifiedDataType	06	OID 0.4.0.127.0.7.3.7.1.1
operationType	80	"FinishTransaction"
clientID	81	"P915560150"
processData	82	"Beleg^0.00_1.65_...^1.65:Bar"
processType	83	"Kassenbeleg-V1"
transactionNumber	85	01E542 (=124226)
serialNumber	04	SHA256(Public Key), 32 Byte
signatureAlgorithm	30	OID 0.4.0.127.0.7.1.1.4.1.4 (SHA384)
signatureCounter	02	05EAF2 (=387826)
logTime (unixTime)	02	635F6B66 (=1 667 197 798)

Tabelle 2: Aufbau der Log-Message (TLV). Ganzzahlen sind als minimale Big-Endian-Bytefolge kodiert.

Zeitstempel. Als `logTime` geht die *End-/Logzeit* 2022-10-31T06:29:58Z ein, umgerechnet in den Unix-Timestamp $1\,667\,197\,798 = 0x635F6B66$ (4 Byte).

Die vollständige Log-Message lautet (166 Byte):

```

1 020102060904007F000703070101801146696E6973685472616E736163
2 74696F6E810A50393135353630313530822742656C65675E302E30305F
3 312E36355F302E30305F302E30305F302E30305E312E36353A42617283
4 0E4B617373656E62656C65672D5631850301E54204202DE151380B9121
5 3FBADFAF18EF856DB499E608283984F96297213F450BA552B7300C060A
6 04007F00070101040104020305EAF20204635F6B66

```

Code Listing 3: Log-Message M (hex)

Der für die Signatur maßgebliche Hashwert ist $h := \text{SHA384}(M)$:

```

1 h = B61488218E6FC197DB055B1ADD7E99372927CE608159054C0833C692
2   DE056AE98B386EA5BFC3DC5DB6FE5C6C92C1F67A

```

Code Listing 4: Hashwert $h = \text{SHA384}(M)$

Da Hash- und Kurvenbitlänge gleich sind (384 Bit), geht h ohne Kürzung als Zahl in die ECDSA-Verifikation ein.

c) Bestimmung der elliptischen Kurve

Die Kurve ist im QR-Code *nicht* angegeben. Wegen der 384-Bit-Koordinaten (r, s, D_x, D_y je 48 Byte) und SHA-384 kommen die gängigen 384-Bit-Kurven in Frage. Detektion durch Einsetzen von $D = (D_x, D_y)$ in die Kurvengleichung $y^2 \equiv x^3 + ax + b \pmod{p}$:

Kurve	$G \in E$	$D \in E$
secp384r1	ja	nein
brainpoolP384r1	ja	ja

Tabelle 3: Test, ob der Public-Key-Punkt D auf der jeweiligen Kurve liegt.

Nur auf **brainpoolP384r1** (OID 1.3.36.3.3.2.8.1.1.11) erfüllt D die Kurvengleichung exakt; secp384r1 scheidet aus. Damit ist die verwendete Kurve eindeutig. Die Parameter:

```

1 p = 8CB91E82A3386D280F5D6F7E50E641DF152F7109ED5456B412B1DA197
2   FB71123ACD3A729901D1A71874700133107EC53
3 a = 7BC382C63D8C150C3C72080ACE05AFA0C2BEA28E4FB22787139165EFB
4   A91F90F8AA5814A503AD4EB04A8C7DD22CE2826
5 b = 04A8C7DD22CE28268B39B55416F0447C2FB77DE107DCD2A62E880EA53
6   EEB62D57CB4390295DBC9943AB78696FA504C11
7 Gx = 1D1C64F068CF45FFA2A63A81B7C13F6B8847A3E77EF14FE3DB7FCAFE0
8   CBD10E8E826E03436D646AAEF87B2E247D4AF1E
9 Gy = 8ABE1D7520F9C2A45CB1EB8E95CFD55262B70B29FEEC5864E19C054FF
10   99129280E4646217791811142820341263C5315
11 n = 8CB91E82A3386D280F5D6F7E50E641DF152F7109ED5456B31F166E6CA
12   C0425A7CF3AB6AF6B7FC3103B883202E9046565

```

Code Listing 5: Parameter der Kurve brainpoolP384r1

d) Verifikation der Signatur

Mit h , dem Generator G , dem Public Key D und (r, s) wird die ECDSA-Verifikation auf brainpoolP384r1 gerechnet:

1. Vorprüfung: $1 \leq r, s \leq n - 1$ ✓ und $D \in E$ ✓.
2. $w := s^{-1} \bmod n$.
3. $u_1 := hw \bmod n$, $u_2 := rw \bmod n$.

$$4. L := [u_1]G + [u_2]D = [s^{-1}]([h]G + [r]D) \in E.$$

5. Gültig, falls $L_x \equiv r \pmod{n}$.

Kern der Verifikation (Punktarithmetik über Bouncy Castle):

```
1 BigInteger w = s.modInverse(n);
2 BigInteger u1 = h.multiply(w).mod(n);
3 BigInteger u2 = r.multiply(w).mod(n);
4 ECPoint L = G.multiply(u1).add(D.multiply(u2)).normalize(); // [s^-1]([h]G + [r]D)
5 boolean valid = L.getAffineXCoord().toBigInteger().mod(n).equals(r);
```

Code Listing 6: ECDSA-Verifikation mit Bouncy Castle

Berechnetes Ergebnis:

```
1 Lx mod n = 6FA9BBAF63C0944AE393D737D1CE1B90F8ABA6A372DB486CE352
2          EB26ECA876EA1D63BDCF0750FA26F411AD1359FA3E0F
3 r        = 6FA9BBAF63C0944AE393D737D1CE1B90F8ABA6A372DB486CE352
4          EB26ECA876EA1D63BDCF0750FA26F411AD1359FA3E0F
```

Code Listing 7: Ergebnis der Verifikation

Es gilt $L_x \bmod n = r$. Die **Signatur ist gültig**: Der Bon gehört nachweislich zu der in der TSE gespeicherten, unveränderten Transaktion, und die QR-Daten (Betrag, Zeit, Zähler) wurden nicht manipuliert.

e) Eigener (selbst erzeugter) Bon

Da kein eigener QR-Bon vorlag, wurde – wie in der Aufgabe erlaubt – ein vollständiger Bon *selbst* erzeugt und mit identischem Verfahren verifiziert (alles auf brainpoolP384r1, reproduzierbar über festes Schlüsselpaar d und festen Nonce k):

1. Schlüsselpaar erzeugen: privater Schlüssel d zufällig, $D := [d]G$.
2. Fiktiver Bon (Betrag 3,50, Kassenbeleg-V1, Bar) $\rightarrow$ Log-Message $M' \rightarrow h' := \text{SHA384}(M')$.
3. Signieren: k zufällig, $r' := ([k]G)_x \bmod n$, $s' := k^{-1}(h' + r'd) \bmod n$.
4. Verifizieren wie in d).

```
1 d = 1154E898CC38E41B6CB83558DE410C8B7591733C01BF36040BD94BE72
2   6960FF471B7C3C829AF6E589C97DE2F2F3B6D9B
3 Dx = 5E8AB01DA3DE7DBA9D9272A3642A7804CCEC8BBD609D3FA0071BD3C0D
4   BA6525CA4F908EF503150C5495A8C16DC7A3D3B
5 Dy = 87F590A64BD1F142D242BF201D21A062AF988A2458EED392E002DC86D
6   14951118250BD868A98A5518358EE3A62009D3C
7 h' = 94005AB96774849DC546943C8C0813C7AFF86A17F0AAC16844BCDC91
8   4BF5C75DF93E40FAF8F911DF099FC807EF2A8BA0
9 r' = 6BB079330987D742215815E77A7E0A315BB1AA1DF5F5F1003E415019
```

```
10 27FB3EB5A503AC1E8152EE48BF8A97BF489884DB
11 s' = 2E95E1EFB2FCB41354EB37FCDE6AD256E5A6FC0F8C187098CE355474
12 5FB8C847B5B6D7B6A42BE842CD1321EBC724AB89
```

Code Listing 8: Selbst erzeugter Bon – Schlüssel, Hash und Signatur

Verifikation: $D \in E \checkmark$ und $L_x \bmod n = r' \checkmark \Rightarrow$ **Signatur gültig**. Der vollständige Signier-/Prüf-Durchlauf bestätigt das Verfahren end-to-end.

Bewertung

- Alle fünf Teilschritte wurden *ohne App* nachvollzogen; die TSE-Signatur des vorgegebenen Bons ist **gültig**.
- Die unabhängige Probe über die Seriennummer (SHA-256 des Public Keys = Aufdruck) bestätigt schon vor der eigentlichen Signaturprüfung die Integrität von QR und Bon.
- Kritisch für die Verifikation waren zwei Punkte: das *vollständige* Befüllen der Log-Message inkl. `transactionNumber` (Tag `0x85`) – fehlt ein Feld, ändert sich der Hash und die Prüfung schlägt fehl – sowie die richtige Kurvenwahl (brainpoolP384r1 statt secp384r1), die sich eindeutig über die Punktprobe $D \in E$ ermitteln lässt.
- Das selbst erzeugte Beispiel zeigt, dass mit denselben Bausteinen (Hash, EC-Arithmetik, ECDSA) beliebige Bons signiert und geprüft werden können.